

PVTOOLS - Changes Log

Fixes to get the site running and to make the PVcopilot degradation pipeline robust.

Each item: what broke -> why -> how it was fixed.

1. Getting the site to run (environment)

Python environment

Broken: nothing would install or run.

Why: system Python 3.9 was too old for scikit-learn 1.7.2; Python 3.12 too new for the pinned pandas 2.0.3 / numpy 1.26.4 (no wheels -> build failures).

Fix: created a conda env 'pvtools' on Python 3.11, which satisfies every pinned dependency; installed requirements.txt.

App crashed on startup - OpenAI key (field_chat.py, pages/pv_image.py)

Broken: import-time crash; site wouldn't boot.

Why: the OpenAI client is constructed at import using OPENAI_API_KEY; with no key it raised immediately.

Fix: created a git-ignored .env and loaded it (real keys added later).

App crashed on startup - ASHRAE S3 download (utils/pvtoolslib.py)

Broken: the entire site refused to boot.

Why: pvtoolslib downloads 2017DesignConditions_s.xlsx.csv from a PRIVATE S3 bucket at import time; with no AWS creds it raised, and since index.py imports every page, it took down the whole app.

Fix: wrapped the download in try/except with an empty-DataFrame fallback. Site boots; only the String Length Calculator's ASHRAE lookup is degraded until the file is provided locally.

'.env' not being read / wrong key winning (index.py)

Broken: keys in .env were ignored; a stale BLOCKED CBORG key from the shell won instead.

Why: page imports read their keys before any load_dotenv() ran, and plain load_dotenv() does not override variables already set in the shell.

Fix: added load_dotenv(override=True) at the very top of index.py, so .env loads first and beats stale shell variables.

Provider switching (CBORG vs OpenAI) (field_chat.py, analysis_utils.py)

Why: wanted to use the LBL CBORG gateway now and switch to OpenAI easily.

Fix: added an LLM_PROVIDER toggle in .env that selects the client and the correct model-name prefix (CBORG needs 'openai/'). Switching is a one-line .env change, no code edits.

2. PVcopilot data-pipeline bugs

Graphs all stuck at Jan 1, 1970 (analysis_utils.py: parse_contents)

Broken: the raw-data preview crammed every point onto 1970; the time axis was destroyed (data values were fine).

Why: uploaded CSVs get a plain integer index 0,1,2,...; the code ran pd.to_datetime() on it, which reads integers as NANOSECONDS SINCE 1970, accepted that as 'time', and discarded the real measured_on column.

Fix: (a) do NOT datetime-convert a numeric index (guard with is_numeric_dtype); (b) promote the identified Time column to the datetime index, since all downstream logic keys off df.index.

'Apply filters' loaded forever (analysis_utils.py; consequence of the 1970 bug)

Broken: on multiyear data, the filter step hung indefinitely.

Why: the 1970 timestamps were distinct only at the nanosecond level; the JSON store round-trip (millisecond precision) collapsed them into IDENTICAL index labels, so df.loc[...] did a many-to-many match and exploded 3,589 rows -> 7,798,897, choking the filters, store, and browser plot.

Fix: resolved by the time fix above - real distinct timestamps survive the round-trip, so no duplicate labels and no explosion.

Degradation crashed: 'Grouper and axis must be same length' (analysis_utils.py: aggregate_

Broken: any real data with gaps errored at the daily-aggregation step.

Why: it called .dropna() (removing rows) but then grouped by df.index.date - the full, original-length index - so the grouper was longer than the data.

Fix: group by the post-dropna frame's OWN index so lengths always match.

YoY returned nan (analysis_utils.py: compute_yoy)

Broken: year-over-year produced no number on irregular data.

Why: it required a data point on the EXACT same calendar date one year earlier; with irregular sampling that almost never exists, so the ratio list was empty -> median([]) = nan.

Fix: pair each point with the NEAREST point within +/-10 days of one year earlier (get_indexer(method='nearest', tolerance=10 days)).

3. PVcopilot now adapts to the data provided (new capability)

Broken: with no irradiance column the tool returned 'Irradiance column not found' and ran nothing - even with valid power+time.

Why: the filter and degradation callbacks did a hard 'if irra_key is None: return error', and normalize() used hard dict lookups for Irradiance / Module temperature (KeyError when missing).

Fix: normalize() -> 3-way adaptive: temp-corrected (irr+temp) / irradiance-only / raw power.
aggregate_daily() -> irradiance-weighted when available, else daily mean. parse_contents() -> compute power = $V \times I$ when only voltage+current exist. Callbacks -> require only power+time; auto-skip clear-sky/low-irradiance filters with no irradiance; show warnings; block clearly only when truly impossible.

Data-tier behavior matrix

- power + time -> YoY/LR/HW/ARIMA/CSD on raw daily power (warn: not normalized)
- voltage + current (no power) -> compute power = $V \times I$, then as above
- power + irradiance -> irradiance-normalized (warn: no temp correction)
- power + irradiance + temp -> full temperature-corrected normalization
- voltage + current + irradiance + temp (+metadata) -> PVPRO enabled
- none of power / voltage+current -> blocked with a clear warning

4. Supporting tooling and test data (not app fixes)

- test_datasets_report.py - runs each file through column-ID, writes a GOOD/WARNING/ERROR PDF.
- fetch_pvdaq_datasets.py - downloads varied raw CSVs from the public OEDI PVDAQ data lake.
- build_usable_datasets.py - builds multi-year/multi-day hourly datasets covering each tier.
- test_datasets/ - ~73 real OEDI PVDAQ datasets spanning every tier/fix (see its README).

Files changed (app)

- index.py - load_dotenv(override=True) at top.
- utils/pvtoolslib.py - ASHRAE S3 download fallback.
- page_supporting_files/field_chat.py - LLM_PROVIDER toggle + model prefix.
- page_supporting_files/analysis_utils.py - LLM_PROVIDER toggle; time-index fix + $V \times I$ in parse_contents; adaptive normalize; robust aggregate_daily; nearest-tolerance compute_yoy.
- pages/pvcopilot.py - adaptive filter + degradation callbacks (relaxed irradiance, warnings, PVPRO check).
- .env - keys + LLM_PROVIDER (git-ignored; not committed).